

Shell Scripting, Cron Jobs, Web Servers & Log Management

1. Shell Scripting

Shell scripting is the practice of writing commands in a text file so that a shell such as Bash can execute them automatically. Scripts are widely used in DevOps for provisioning, backups, deployments, system administration, monitoring, and automation. A Bash script normally starts with a shebang such as `#!/bin/bash` and can include variables, conditions, loops, functions, and commands.

Example:

```
#!/bin/bash
echo "Starting backup..."
mkdir -p /opt/backups
echo "Backup directory ready."
```

Using `set -euo pipefail` is a common practice for more reliable Bash scripts. It helps detect command failures, unset variables, and failures inside pipelines.

2. Variables

Variables store values that can be reused throughout a script. They make scripts configurable instead of hardcoding the same values repeatedly. In Bash, a variable is assigned without spaces around the equals sign and accessed using a dollar sign.

```
GROUP="devops"
BASE_DIR="/opt/devops"
echo "Group: $GROUP"
echo "Directory: $BASE_DIR"
```

Variables can contain strings, numbers, command output, and paths. Positional parameters such as `$1`, `$2`, and `$@` allow scripts to accept values from the command line.

3. Loops

Loops repeat commands. Bash provides `for`, `while`, and `until` loops. A `for` loop is especially useful for processing lists of users, files, directories, or servers.

```
USERS=("developer1" "developer2" "developer3")
```

```
for USER in "${USERS[@]}"; do
echo "Processing $USER"
done
```

Loops reduce repetition and make automation scripts easier to maintain.

4. Functions

Functions group related commands into reusable blocks. They improve readability and prevent the same logic from being copied throughout a script. Functions can receive arguments and return a status code.

```
log() {
echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*"
}
log "Backup started"
```

For larger DevOps scripts, functions are useful for argument validation, logging, user creation, backup operations, and error handling.

5. Cron Jobs

Cron is a Linux scheduling service used to execute commands or scripts automatically at specified times. A user's scheduled tasks are configured with `crontab -e`. System-level scheduled jobs can also be configured through system cron directories.

A cron expression contains five time fields followed by the command:

minute hour day-of-month month day-of-week

For example, the following runs a backup every day at 2:00 AM:

```
0 2 * * * /home/vagrant/scripts/backup.sh
```

For testing, a job can be scheduled every five minutes using `*/5 * * * *`. Absolute paths should be used in cron jobs because cron runs with a limited environment compared with an interactive terminal.

6. Apache

Apache HTTP Server is a widely used open-source web server. It can serve static files and support dynamic web applications through modules and integrations. Apache uses configuration files and

virtual hosts to serve multiple websites from one server. Typical Apache responsibilities include serving websites, handling HTTP requests, TLS/HTTPS configuration, access control, URL rewriting, and application integration.

Example Apache workflow: Client → Apache → Application / PHP / Static Files → Database

7. Nginx

Nginx is an open-source web server and reverse proxy. It is commonly used for serving static content, terminating TLS, routing requests to backend applications, and load balancing. A common DevOps architecture places Nginx in front of an application server. For example, requests to / can be served by a frontend while requests to /api are forwarded to a Node.js backend.

Typical architecture: Internet → Nginx → Frontend / Backend Application → Database

8. Log Management

Logs record events generated by operating systems, applications, web servers, databases, and automation scripts. They are essential for troubleshooting, security monitoring, auditing, and understanding system behavior. Linux commonly stores system and service logs under /var/log. Examples include authentication logs, system logs, and web-server access/error logs. Modern Linux systems may also use systemd-journald, which can be queried with journalctl.

Useful commands

```
tail -f /var/log/application.log
```

```
grep "ERROR" /var/log/application.log
```

```
journalctl -u nginx
```

```
journalctl -u cron
```

Application scripts can also maintain their own logs. A useful logging function includes a timestamp and a meaningful message. For example, a backup script might record when a backup starts, completes, or fails.

Basic DevOps Logging Practices

- Use timestamps for important events.
- Record successful and failed operations.
- Avoid logging passwords, API keys, tokens, or other secrets.
- Use log rotation and retention policies so logs do not fill the disk.
- Centralize logs when managing multiple servers or containers.
- Monitor logs for repeated failures and security-related events.